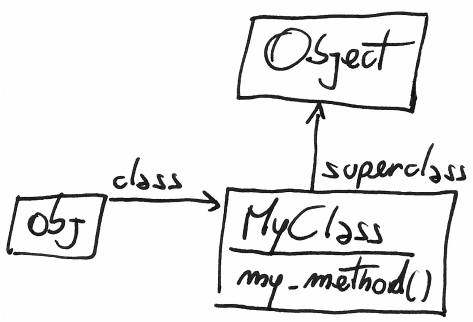
```
obj = MyClass.new
obj.my_method
```

Bill draws the following flowchart and says, "When you call my_method, Ruby goes right into MyClass and finds the method there." So far, so good.



Now, what happens if you define a *Singleton Method (114)* on obj?

```
def obj.my_singleton_method; end
```

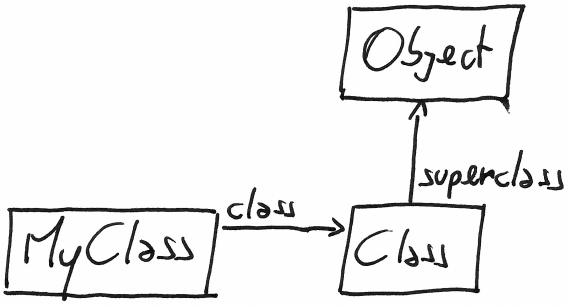
If you look at the previous flowchart, you'll notice that there's no obvious home for my_singleton_method there.

The Singleton Method can't live in obj, because obj is not a class. It can't live in MyClass, because if it did, all instances of MyClass would share it. And it cannot be an instance method of MyClass's superclass, Object. So then, where do Singleton Methods live?

Class methods are a special kind of Singleton Method—and just as baffling:

```
def MyClass.my_class_method; end
```

If you look at the following figure, you'll find that, again, my_class_method doesn't seem to live anywhere in Bill's diagram.



The explanation of this mystery could surprise you.